

Day 6 — Routing & Forms

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Give the app pages with React Router (nested routes, params, navigation) and build validated forms with React Hook Form — a small HR Portal.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Welcome to Day 6. Today we give our app **pages** and **forms**.

Two big ideas:

1. **Routing** — showing different screens for different URLs (like `/employees` and `/employees/5`) *without* reloading the whole page.
2. **Forms** — collecting typed-in data (name, email...) and checking it is valid before we save.

New word: a **route** is just a rule that says "when the URL looks like *this*, show *that* component." Think of it like a page in your app.

New word: **SPA** (Single Page Application). React apps load one HTML page once, then swap the content with JavaScript. React Router is the tool that swaps that content based on the URL.

We build toward a small **HR Portal**: a shared layout with a nav bar -> an employees list -> an employee detail page (`/employees/:id`) -> an add/edit employee form with validation.

Quick .NET picture: React Router is like your **ASP.NET route table / attribute routing** (`[Route("employees/{id}")]`), and React Hook Form validation is like **model validation / data annotations** (`[Required]` , `[EmailAddress]`) on a C# model.

Part A — Core Topics

1. Setting up React Router

First install it: `npm install react-router-dom`. Then wrap your whole app in `<BrowserRouter>`. Inside, `<Routes>` holds a list of `<Route>` rules. Each `<Route>` maps a `path` (the URL) to an `element` (the component to show).

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="employees" element={<Employees />} />
      </Routes>
    </BrowserRouter>
  );
}
```

- **.NET / KnockoutJS:** Like registering routes in `RouteConfig` / `MapControllerRoute` — a table of "URL pattern -> what handles it."
 - **You'll be able to...** show different components for different URLs.
-

2. Link and NavLink

To move between routes, use `<Link>` instead of a normal `<a>` tag. An `<a>` reloads the whole page; `<Link>` just swaps the content (fast, keeps your state). `<NavLink>` is a `Link` that also knows if *it* is the current page, so you can style the active menu item. Its `className` can be a function that receives `{ isActive }`.

```
import { Link, NavLink } from "react-router-dom";

function Nav() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <NavLink
        to="/employees"
        className={({ isActive }) => (isActive ? "active" : "")}
      >
        Employees
      </NavLink>
    </nav>
  );
}
```

- **.NET / KnockoutJS:** Like `@Html.ActionLink` / `Url.Action`, but with no full page reload (client-side navigation).
 - **You'll be able to...** build a nav bar that highlights the page you're on.
-

3. Nested routes + Outlet

Often many pages share the same frame (a top nav, a footer). Make a **layout** route whose element renders the shared parts plus an `<Outlet />`. New word: **Outlet** is a placeholder — the child route's component renders *inside* it. Child `<Route>`s are nested inside the parent `<Route>`. The `index` route is the default child shown at the parent's own path.

```
import { Routes, Route, Outlet } from "react-router-dom";

function Layout() {
  return (
    <div>
      <Nav />
      <Outlet /> { /* the matching child route renders here */ }
    </div>
  );
}

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="employees" element={<Employees />} />
      </Route>
    </Routes>
  );
}
```

- **.NET / KnockoutJS:** Like a `_Layout.cshtml` master page; `<Outlet />` is where `@RenderBody()` would go.
- **You'll be able to...** share one nav/layout across many pages without repeating it.

4. Route params

A **route param** is a changing piece of the URL, written with a colon: `employees/:id`. So `/employees/5` and `/employees/42` hit the same route, and you read the value with `useParams`. Important for TypeScript: `useParams` gives you `string | undefined` (the value might be missing), so handle the `undefined` case.

```
import { useParams } from "react-router-dom";

function EmployeeDetail() {
  const { id } = useParams<{ id: string }>(); // id: string | undefined
  if (!id) return <p>No employee selected.</p>;
  return <p>Showing employee {id}</p>;
}

// route: <Route path="employees/:id" element={<EmployeeDetail />} />
```

- **.NET / KnockoutJS:** Exactly like `[Route("employees/{id}")]` and reading `int id` from the action parameter.
- **You'll be able to...** build a detail page that works for any employee id in the URL.

5. Programmatic navigation

Sometimes *code* needs to change the page — for example, after you save an employee, jump to their detail page. `useNavigate` gives you a `navigate` function. Call `navigate("/some/path")` to go there, or `navigate(-1)` to go back.

```
import { useNavigate } from "react-router-dom";

function SaveButton() {
  const navigate = useNavigate();

  function handleSave() {
    // ... save the employee ...
    navigate("/employees/42"); // then go to the detail page
  }

  return <button onClick={handleSave}>Save</button>;
}
```

- **.NET / KnockoutJS:** Like `return RedirectToAction("Detail", new { id })` after a successful save.
- **You'll be able to...** redirect the user in code (e.g. after saving or logging in).

6. Query params

Query params are the `?key=value` part of a URL, like `/employees?search=anna&page=2`. They're great for search boxes, filters, and paging (from Day 5). `useSearchParams` works like `useState`: you read with `.get("search")` and update with `setSearchParams`. Bonus: the URL becomes shareable/bookmarkable.

```
import { useSearchParams } from "react-router-dom";

function EmployeeList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const search = searchParams.get("search") ?? ""; // ?? handles null

  return (
    <input
      value={search}
      onChange={(e) => setSearchParams({ search: e.target.value })}
    />
  );
}
```

- **.NET / KnockoutJS:** Like reading `Request.Query["search"]` or a `[FromQuery] string search` parameter.
- **You'll be able to...** put search/page state in the URL so it can be shared and refreshed.

7. A not-found (404) route

If the URL matches no route, you should show a friendly "page not found" instead of a blank screen. The special path `"*"` means "match anything else." Put it **last** so it only catches leftovers.

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="employees" element={<Employees />} />
      <Route path="*" element={<NotFound />} /> { /* catch-all */ }
    </Routes>
  );
}
```

- **.NET / KnockoutJS:** Like a catch-all route or a custom 404 error page.
- **You'll be able to...** show a proper "Not Found" page for unknown URLs.

8. Forms with React Hook Form

Install it: `npm install react-hook-form`. `useForm` is a hook that manages your whole form for you. New word: **register** connects an input to the form (you spread `{ ...register("name")}` onto the input). `handleSubmit` wraps your submit function — it runs validation first, and only calls you with the data if the form is valid. Errors live in `formState.errors`. Type your form with an interface so `data` is fully typed.

```
import { useForm } from "react-hook-form";

interface EmployeeForm {
  name: string;
  email: string;
}

function AddEmployee() {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<EmployeeForm>();

  const onSubmit = (data: EmployeeForm) => {
    console.log(data); // fully typed
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input { ...register("name")} />
      {errors.name && <span>{errors.name.message}</span>}
      <button type="submit">Save</button>
    </form>
  );
}
```

- **.NET / KnockoutJS:** Like binding a form to a C# **view model** (or a Knockout `viewModel` with `data-bind`), then posting the whole model at once.
- **You'll be able to...** collect typed form data with very little code.

9. Validation (required, minLength, pattern, custom validate)

You pass **rules** as the second argument to `register`. Common ones: `required`, `minLength`, `maxLength`, `pattern` (a regex), and `validate` (your own function returning `true` if OK, or an error string if not). Give each rule a message string — that message shows up in `errors.<field>.message`.

```

import { useForm } from "react-hook-form";

interface EmployeeForm {
  name: string;
  email: string;
}

function AddEmployee() {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<EmployeeForm>();

  return (
    <form onSubmit={handleSubmit((d) => console.log(d))}>
      <input
        { ...register("name", {
          required: "Name is required",
          minLength: { value: 2, message: "Name is too short" },
        })}
      />
      {errors.name && <span>{errors.name.message}</span>}

      <input
        { ...register("email", {
          required: "Email is required",
          pattern: { value: /^\S+@\S+$/ , message: "That is not a valid email" },
          validate: (value) =>
            value.endsWith("@company.com") || "Must be a work email",
        })}
      />
      {errors.email && <span>{errors.email.message}</span>}

      <button type="submit">Save</button>
    </form>
  );
}

```

- **.NET / KnockoutJS:** Exactly like **data annotations** — `[Required]` , `[StringLength(2)]` , `[RegularExpression]` , `[EmailAddress]` — plus custom `IDataValidatableObject` for the `validate`

case.

- **You'll be able to...** block bad input and show clear error messages next to each field.

Part B — Extra Topics

(These are the only "extra" additions to Day 6. Everything above is the core plan.)

Extra 1: Controlled inputs (Day 4) vs React Hook Form

On Day 4 we used **controlled inputs**: every keystroke updates `useState`, which re-renders the component each time. That's fine for small forms, but it's a lot of boilerplate and lots of re-renders on big forms.

React Hook Form uses **uncontrolled** inputs instead. New word: **uncontrolled** means React doesn't hold the value in state — the input keeps its own value, and RHF reads it through a **ref** (a direct pointer to the real DOM input). Result: the component barely re-renders as you type, and you write far less code.

```
import { useState } from "react";
import { useForm } from "react-hook-form";

// Day 4 way: controlled - re-renders on EVERY keystroke
function ControlledName() {
  const [name, setName] = useState("");
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}

// Day 6 way: React Hook Form - uncontrolled (uses a ref), fewer re-renders
function RhfName() {
  const { register } = useForm<{ name: string }>();
  return <input {...register("name")} />;
}
```

Rule of thumb: use RHF for real forms; keep controlled inputs for tiny one-off fields or when you truly need the value on every keystroke.

Extra 2: Schema validation with zod (light intro)

Instead of writing rules inline on each field, you can describe the **whole shape** of your form in one place with **zod** (a schema/validation library). New word: a **schema** is a single object that says what valid data looks like. `@hookform/resolvers` connects that schema to React Hook Form via a **resolver** (the bridge that runs your schema for RHF).

Install: `npm install zod @hookform/resolvers`.

```

import { z } from "zod";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";

const schema = z.object({
  name: z.string().min(2, "Name is too short"),
  email: z.string().email("That is not a valid email"),
});

type EmployeeForm = z.infer<typeof schema>; // TS type comes FROM the schema

function AddEmployee() {
  const { register, handleSubmit } = useForm<EmployeeForm>({
    resolver: zodResolver(schema),
  });

  return <form onSubmit={handleSubmit((d) => console.log(d))}>{/* fields */}</form>;
}

```

Nice bonus: `z.infer` gives you the TypeScript type for free, so the schema and the type can never drift apart. (We keep zod light today — inline rules are perfectly fine to start.)

Extra 3: Form accessibility basics

Accessibility means making the form usable for everyone, including screen readers. Two simple habits:

1. Give every input a `<label>` whose `htmlFor` matches the input's `id` (clicking the label focuses the input). Note: in React it's `htmlFor`, not `for`.
2. Tie the error text to the input with `aria-invalid` and `role="alert"` so screen readers announce it.

```
function EmailField({ register, error }: EmailFieldProps) {
  return (
    <div>
      <label htmlFor="email">Email</label>
      <input
        id="email"
        {...register("email", { required: "Email is required" })}
        aria-invalid={error ? "true" : "false"}
      />
      {error && (
        <span role="alert">{error}</span>
      )}
    </div>
  );
}
```

(In real code `register` and `error` come from `useForm` in the parent; here they're just passed in as props to keep the example focused.)

- **.NET / KnockoutJS:** Like using `@Html.LabelFor` + `@Html.ValidationMessageFor`, which wire the label and error to the field for you.

Extra 4: Heads-up — what comes later

- Turning these inputs, buttons, and the form into **reusable components** (an `Input`, a `Button`, a `Form`) is **Day 7**.
- **Global/shared state** across pages (React Context, reducers) is **Day 8**. Today, route params and query params already carry a lot of "state" in the URL for us.

Extra 5: .NET contrast recap (keep this mental map)

React (Day 6)	.NET / ASP.NET
<code><Route path="employees/:id"></code>	Route table / <code>[Route("employees/{id}")]</code>
<code>useParams</code>	Action parameter <code>int id</code>
<code>useSearchParams</code>	<code>[FromQuery] string search</code>
React Hook Form + rules	View model + data annotations (<code>[Required]</code> , <code>[EmailAddress]</code>)
<code>validate: (v) => ...</code>	Custom <code>IValidatableObject</code>
zod schema	FluentValidation (one place, whole model)

That's the full Learning Plan for Day 6. Next you'll type these out yourself to build the little HR Portal:
layout + nav -> employees list -> employee detail (/employees/:id) -> add/edit form with validation.

Detailed Notes — Read Once, Understand Everything

The big idea first

A **Single Page App (SPA)** — that means a website that loads **one** HTML page and then never fully reloads again — feels fast because the browser is not fetching a brand new page every time you click. Instead, JavaScript quietly swaps what you see.

Two tools make this pleasant:

1. A **router** — it looks at the URL (the web address in the bar) and decides **which component to show**. So the URL becomes part of your app's state (its "current situation"). Change the URL, the screen changes. This is exactly like **ASP.NET routing / MVC routes** in C#, where a URL like `/Employees/Details/5` maps to a controller action. Same idea — but here it happens in the browser, with no round trip to the server.
2. A **form library** — forms collect what the user types. Writing forms by hand gets long and repetitive. **React Hook Form** makes it short, fast, and easy to validate.

That is the whole day. Now the details, one small piece at a time.

1. What client-side routing is (and why an SPA needs a router)

In an old-style website (think classic ASP.NET MVC returning full views), every link click asks the **server** for a brand new HTML page. The screen goes blank for a moment, then the new page paints. All your in-memory state is gone.

In an SPA, we do **client-side routing** — "client-side" just means "in the browser, using JavaScript." When you click a link, the router:

- changes the URL (using the browser History API — the thing that powers the Back button),
- picks the matching component,
- and renders it — **without** asking the server for a new page.

No blank flash. No lost state. That is why an SPA needs a router: to fake the "many pages" feeling while really staying on one page.

C# comparison: your `RouteConfig` / attribute routes said "this URL → this action." A React router says "this URL → this component." Same job, different place.

2. Setting up: `BrowserRouter` once at the top

You wrap your whole app **one time** in `<BrowserRouter>`. Think of it as "turn routing on for everything inside." (`BrowserRouter` uses clean URLs like `/employees/5`, powered by the History API.)

```
import { BrowserRouter } from "react-router-dom";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Then, somewhere inside, you list your routes with `<Routes>` (the container) and `<Route>` (one rule each). `path` is the URL to match; `element` is what to show.

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/employees" element={<EmployeeList />} />
    </Routes>
  );
}
```

`<Routes>` looks at the current URL and renders the **one** `<Route>` that matches best. It is like a `switch` statement over the URL.

3. `Link` / `NavLink` — never use a plain `<a>` for internal links

To move between pages **inside** your app, use `<Link>`, not `<a href>`.

Why? A plain `<a href="/employees">` tells the **browser** to go fetch that address from the server — a full reload, blank flash, lost state. That kills the whole point of an SPA. `<Link>` intercepts the click and lets the router handle it in the browser.

```
import { Link } from "react-router-dom";

<Link to="/employees">View employees</Link>
```

`<NavLink>` is a `<Link>` with a bonus: it knows when **its own** link is the current page ("active"). Great for nav menus where you want to highlight the current tab. It hands you an `isActive` flag:

```
import { NavLink } from "react-router-dom";

<NavLink
  to="/employees"
  className={({ isActive }) => (isActive ? "active" : "")}
>
  Employees
</NavLink>
```

Rule of thumb: **internal** navigation → `Link` / `NavLink`. A link to a truly **external** site (like `https://google.com`) → a normal `<a>` is fine, because you really do want to leave.

4. Nested routes + `Outlet` — a shared layout

Most apps have a shared shell: the same header and nav on every page, with only the middle part changing. You build that with a **layout route** — a parent `<Route>` whose `element` contains the shared stuff plus an `<Outlet />`.

`<Outlet />` means "render the matching child route right here." It is the hole where the child page drops in.

```

import { Routes, Route, Outlet, NavLink } from "react-router-dom";

function Layout() {
  return (
    <div>
      <header>
        <nav>
          <NavLink to="/">Home</NavLink>
          <NavLink to="/employees">Employees</NavLink>
        </nav>
      </header>
      <main>
        { /* the current child page shows up here */ }
        <Outlet />
      </main>
    </div>
  );
}

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="employees" element={<EmployeeList />} />
        <Route path="employees/:id" element={<EmployeeDetail />} />
      </Route>
    </Routes>
  );
}

```

Two things to notice:

- Child `path` values are **relative** (no leading `/`): `employees`, not `/employees`.
- The `index` route is the child that shows when the URL exactly matches the parent (here, `/`). "Index" just means "the default child."

C# comparison: this is like a shared `_Layout.cshtml` where `@RenderBody()` marks the spot the specific view fills in. `<Outlet />` is React's `@RenderBody()`.

5. Route params — `/employees/:id`

A **route param** is a placeholder in the path. The `:id` means "any value goes here, call it `id`." So `/employees/5` and `/employees/99` both match `employees/:id`.

You read it with `useParams()`. Important for TypeScript: the value is `string | undefined` — a string because URLs are text, and possibly `undefined` because TypeScript can't promise the segment is really there. So you must handle the `undefined` case.

```
import { useParams } from "react-router-dom";

function EmployeeDetail() {
  const { id } = useParams<{ id: string }>();
  // id has type: string | undefined

  if (!id) {
    return <p>No employee id in the URL.</p>;
  }

  const employeeId = Number(id); // URL text → number for lookups
  return <p>Showing employee #{employeeId}</p>;
}
```

Two habits to keep:

- Always guard the `undefined` (the `if (!id)` above).
- URL params are **text**, so convert with `Number(id)` when you need a number.

6. Programmatic navigation — `useNavigate()`

Sometimes you want to move the user **from code**, not from a click on a link. Example: after saving an employee, jump to that employee's detail page. That is where `useNavigate()` comes in. It gives you a `navigate` function you call with a path.

```
import { useNavigate } from "react-router-dom";

function SaveButton() {
  const navigate = useNavigate();

  function handleSave() {
    const newId = 42; // pretend the server returned this new id
    navigate(`/employees/${newId}`); // go to the detail page
  }

  return <button onClick={handleSave}>Save</button>;
}
```

You can also go **back** like the browser's Back button by passing a number:

```
<button onClick={() => navigate(-1)}>Back</button>
```

C# comparison: `navigate("/employees/42")` is the React version of `return RedirectToAction("Details", new { id = 42 })`.

7. Query params — `useSearchParams` for `?search=&page=`

Query params are the part after the `?` in a URL, like `/employees?search=john&page=2`. They are perfect for filters, search text, and page numbers, because putting them in the URL makes the result **shareable and bookmarkable** — copy the link, and someone else sees the same filtered list.

`useSearchParams` works a lot like `useState`, but the "state" lives in the URL.

```
import { useSearchParams } from "react-router-dom";

function EmployeeList() {
  const [searchParams, setSearchParams] = useSearchParams();

  const search = searchParams.get("search") ?? ""; // ?? = "if null, use this"
  const page = Number(searchParams.get("page") ?? "1");

  function handleSearchChange(e: React.ChangeEvent<HTMLInputElement>) {
    setSearchParams({ search: e.target.value, page: "1" });
  }

  return (
    <input value={search} onChange={handleSearchChange} placeholder="Search ... " />
  );
}
```

Note `.get(...)` returns `string | null`, so we use `?? ""` (the "null-coalescing" operator — same symbol and meaning as C#'s `??`) to give a safe default.

8. Not-found route — `path="*"`

`*` is a wildcard — it means "match anything that no other route matched." Put it **last** so it acts as a catch-all 404 page. Without it, a bad URL just shows nothing.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/employees" element={<EmployeeList />} />
  <Route path="*" element={<NotFound />} /> { /* 404 */ }
</Routes>
```

9. Forms intro — controlled inputs work, but get verbose

On Day 4 you built **controlled inputs**: each input's value lives in `useState`, and every keystroke calls `setState`. That works and gives full control, but for a big form it is a lot of typing — one state variable and one `onChange` per field.

```
// Controlled (Day 4 style) – fine, but repetitive for many fields:
const [name, setName] = useState("");
<input value={name} onChange={(e) => setName(e.target.value)} />
```

React Hook Form takes a different, lighter approach: it uses **uncontrolled inputs** — the input keeps its own value in the DOM (the browser's page structure), and the library reads it with a **ref** (a direct handle to the real input element) only when needed. Result: less code, fewer re-renders, so it stays fast even with big forms.

10. React Hook Form basics — `useForm`, `register`, `handleSubmit`, `errors`

The core pieces:

- `useForm<FormValues>()` — start a form; the `<FormValues>` type describes the fields.
- `register("field")` — connect an input to the form. You spread it onto the input:
`{ ...register("name")}`.
- `handleSubmit(onSubmit)` — wraps your submit handler; it gathers all values and hands them to you as one typed object (and blocks submit if validation fails).
- `formState.errors` — an object holding any validation errors, so you can show messages.

```
import { useForm } from "react-hook-form";

interface EmployeeFormValues {
  name: string;
  email: string;
  age: number;
}

function EmployeeForm() {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<EmployeeFormValues>();

  function onSubmit(data: EmployeeFormValues) {
    console.log(data); // fully typed: data.name, data.email, data.age
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input { ...register("name")} />
      <input { ...register("email")} />
      <button type="submit">Save</button>
    </form>
  );
}
```

Typing the form values with an `interface` is the key step: now `data` is fully typed, and `register("name")` is checked — misspell a field name and TypeScript complains.

11. Validation — rules on `register`, and showing the messages

You add validation by passing a rules object as the **second argument** to `register`. The message you give becomes the error text. Common rules: `required`, `minLength`, `maxLength`, `pattern` (a regex — a text-matching pattern), and `validate` (your own custom check).

```

<input
  { ...register("name", {
    required: "Name is required",
    minLength: { value: 2, message: "At least 2 characters" },
    maxLength: { value: 50, message: "Too long" },
  })}
/>
{errors.name && <span role="alert">{errors.name.message}</span>}

<input
  { ...register("email", {
    required: "Email is required",
    pattern: {
      value: /^S+@\S+\.\S+$/,
      message: "Enter a valid email",
    },
  })}
/>
{errors.email && <span role="alert">{errors.email.message}</span>}

<input
  type="number"
  { ...register("age", {
    valueAsNumber: true, // turn the text into a real number
    required: "Age is required",
    validate: (value) => value ≥ 18 || "Must be 18 or older",
  })}
/>
{errors.age && <span role="alert">{errors.age.message}</span>}

```

`errors.name && <span> ... </span>` just means "if there is an error, show the span." A custom `validate` function returns `true` when OK, or the **message string** when not.

C# comparison: these rules are the React version of **data annotations** like `[Required]`, `[StringLength(50)]`, `[RegularExpression( ... )]` on your model. The rules just live on the input instead of on a C# property.

Optional — schema validation with zod (light mention): instead of scattering rules across inputs, you can define one **schema** (a single description of what valid data looks like) with **zod**, and plug it in with `@hookform/resolvers`. This is nice for bigger forms and lets you reuse the type.

```
import { z } from "zod";
import { zodResolver } from "@hookform/resolvers/zod";
import { useForm } from "react-hook-form";

const schema = z.object({
  name: z.string().min(2, "At least 2 characters"),
  email: z.string().email("Enter a valid email"),
  age: z.number().min(18, "Must be 18 or older"),
});

type EmployeeFormValues = z.infer<typeof schema>; // type derived from the schema

const { register, handleSubmit } = useForm<EmployeeFormValues>({
  resolver: zodResolver(schema),
});
```

Don't worry about mastering zod today — just know it exists as the "one central rulebook" option.

12. Accessibility — labels and linked error text

"Accessibility" means making the form usable for everyone, including people using screen readers (software that reads the screen aloud). Two small habits do most of the work:

1. Pair a `<label htmlFor="id">` with an `<input id="id">`. Now clicking the label focuses the input, and screen readers announce the field's name. (`htmlFor` is JSX's spelling of HTML's `for`.)
2. Link the error message to the input with `aria-describedby`, and mark the input `aria-invalid` when it's wrong. Now the error is read out too.

```
<label htmlFor="name">Name</label>
<input
  id="name"
  {...register("name", { required: "Name is required" })}
  aria-invalid={errors.name ? "true" : "false"}
  aria-describedby="name-error"
/>
{errors.name && (
  <span id="name-error" role="alert">
    {errors.name.message}
  </span>
)}
```

`role="alert"` tells assistive tech "read this out as soon as it appears."

Heads up: making these inputs into **reusable** components (your own `<TextField />`) is Day 7, and app-wide/global state (Context, reducers) is Day 8. Today we keep routes and forms in plain components.

One-line summary

A **router** turns the URL into your app's state — swap components with `Routes` / `Route`, navigate with `Link` / `NavLink` / `useNavigate`, read `:id` with `useParams` and `?filters` with `useSearchParams`, all nested inside a layout's `<Outlet />` — while **React Hook Form** collects and validates input with tiny, fast, fully-typed code.

3 Common Mistakes to Avoid

These are the small slip-ups that trip up almost everyone on their first React Router + forms day. Read them once now, and again after you build the HR Portal. Each one is short: what goes wrong (❌) and the easy fix (✅).

1. Using a plain `<a href>` for internal links

A normal `<a>` tag does a **full page reload** — the browser throws away your whole React app and downloads it again from scratch. Any state you had in memory is gone.

```
// ❌ full page reload - the app restarts, in-memory state is lost
<a href="/employees">Employees</a>

// ✅ client-side navigation - no reload, state stays
<Link to="/employees">Employees</Link>

// ✅ NavLink also tells you when the link is the current page (for highlighting)
<NavLink
  to="/employees"
  className={({ isActive }) => (isActive ? "active" : "")}
>
  Employees
</NavLink>
```

Why it matters: `<Link>` / `<NavLink>` just swap what's on screen. Reserve `<a href>` for links that leave your app (external sites). **.NET friend:** think of it like `@Html.ActionLink( ... )` / `asp-route` — the framework builds the navigation for you instead of a hard link.

2. Forgetting `<Outlet />` in a layout route

A layout route (nav bar + shared shell) needs to say **where the child page goes**. That spot is `<Outlet />`. Forget it, and your child routes match but render nothing — you just see the nav and an empty area.

```
// ❌ nav shows, but child routes have nowhere to render → blank content
function Layout() {
  return (
    <div>
      <nav>{/* links here */</nav>
    </div>
  );
}

// ✅ <Outlet /> is the placeholder where the matched child route appears
function Layout() {
  return (
    <div>
      <nav>{/* links here */</nav>
      <Outlet />
    </div>
  );
}
```

.NET friend: `<Outlet />` is basically `@RenderBody()` in a Razor `_Layout.cshtml`. The layout is the frame; the outlet is the hole the page drops into.

3. Putting a leading `/` on a nested (child) route path

Inside a parent route, child paths are **relative** — they build on the parent. A leading `/` makes the path **absolute**, so it ignores the parent and breaks your URL.

```
// ❌ leading "/" makes it absolute → React Router rejects it: an absolute
//   child path must start with the parent ("/employees/:id")
<Route path="employees" element={<EmployeesLayout />}>
  <Route path="/:id" element={<EmployeeDetail />} />
</Route>

// ✅ child paths are relative – no leading slash
<Route path="employees" element={<EmployeesLayout />}>
  <Route index element={<EmployeeList />} />
  <Route path=":id" element={<EmployeeDetail />} />
</Route>
```

Why it matters: the `<Route index>` is the default page shown at `/employees`, and `:id` correctly becomes `/employees/:id`. **.NET friend:** like route templates in ASP.NET — a child template extends the

parent prefix; you don't repeat the whole path.

4. Not wrapping the app in a single Router

Hooks like `useNavigate`, `useParams`, and `useSearchParams` only work **inside** a Router. If there's no `<BrowserRouter>` around your app, they throw at runtime. Rule: wrap the app **once**, near the very top — never nest two routers.

```
// ❌ no <BrowserRouter> anywhere → runtime error:
//   "useNavigate() may be used only in the context of a <Router> component."
// createRoot(document.getElementById("root")).render(<App />);

// ✅ wrap the whole app ONE time, at the root
createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Why it matters: the Router is the "context" that feeds those hooks. No Router = no context = crash.

5. Forgetting that `useParams()` values are `string | undefined`

Route params are always **strings**, and TypeScript marks them as maybe-`undefined` (the param might not be there). So an `id` from the URL is **not** a number yet — convert it.

```
const { id } = useParams(); // id is: string | undefined

// ❌ TS error: Type 'string | undefined' is not assignable to type 'number'
// const employeeId: number = id;

// ✅ convert the string to a number yourself
const employeeId = Number(id); // e.g. "42" → 42
```

Why it matters: if you compare `id === 42` (string vs number) it's always false, and math on a string misbehaves. Convert first, then use. **.NET friend:** it's like a route value coming in as `string` — you still `int.Parse` / `Convert.ToInt32` before treating it as an `int`.

6. Calling your submit function directly (forgetting the `handleSubmit` wrapper)

React Hook Form gives you `handleSubmit`. It runs your **validation first**, calls `preventDefault()` for you, and only then hands you the clean data. Wire your own `onSubmit` straight to the form and you skip

all of that.

```
const onSubmit = (data: EmployeeFormValues) => saveEmployee(data);

// ❌ no validation, no preventDefault (and a type error – the form event isn't your data)
// <form onSubmit={onSubmit}> ... </form>

// ✅ wrap it – validation runs, then onSubmit gets the typed values
<form onSubmit={handleSubmit(onSubmit)}> ... </form>
```

.NET friend: `handleSubmit` is like the MVC model-binding + `if (ModelState.IsValid)` step happening automatically before your action body runs.

7. Not typing your form values (or route params)

Give `useForm` an interface. You get autocomplete on field names, and typos get caught at compile time instead of at 2 a.m. in production.

```
// ❌ no type → field names aren't checked, no autocomplete, typos slip through
const { register, handleSubmit } = useForm();

// ✅ describe the shape once, get checking + autocomplete everywhere
interface EmployeeFormValues {
  name: string;
  email: string;
  department: string;
}
const { register, handleSubmit } = useForm<EmployeeFormValues>();
```

.NET friend: it's the difference between a strongly-typed **ViewModel** and loose `dynamic` / `ViewBag`. Type it and let the compiler help you.

8. Using `register` on a custom / third-party component

`register("field")` spreads native input props (`name` , `onChange` , `onBlur` , `ref`). A plain `<input>` understands those. A fancy component (a date picker, a custom `<Select>`) usually does **not**, so the binding silently fails — or won't even compile. For those, use `Controller`.

```
// ❌ register's props don't fit a non-native component → won't bind (and may not compile)
// <DatePicker { ... register("joinDate")} />

// ✅ Controller bridges react-hook-form to any custom input
<Controller
  name="joinDate"
  control={control}
  rules={{ required: "Join date is required" }}
  render={({ field }) => (
    <DatePicker value={field.value} onChange={field.onChange} />
  )}
/>
```

Rule of thumb: native `<input>` / `<select>` / `<textarea>` → `register`. Anything custom → `Controller`. (`control` comes from `useForm`.)

9. Not showing validation errors to the user

Validation can run perfectly and the user still sees **nothing**. You have to read `formState.errors` and display the message yourself.

```
const { register, formState: { errors } } = useForm<EmployeeFormValues>();

// ❌ validation runs, but there's no feedback on screen
<input { ...register("email", { required: "Email is required" })} />

// ✅ show the message when the field has an error
<input { ...register("email", { required: "Email is required" })} />
{errors.email && <span className="error">{errors.email.message}</span>}
```

.NET friend: this is your `<span asp-validation-for="Email">` / `@Html.ValidationMessageFor(...)` — the rule exists, but you still have to render the message.

10. Mixing a controlled `value` + `onChange` with `register` on the same field

Pick one owner for a field. If you add your own `value` / `onChange` and `register`, both fight to control the input, and it behaves weirdly (cursor jumps, stale values).

```
const [name, setName] = useState("");

// ❌ two owners fighting: your state AND react-hook-form on one input
// <input value={name} onChange={(e) => setName(e.target.value)} {...register("name")} />

// ✅ let react-hook-form own it - just register
<input {...register("name", { required: "Name is required" })} />
```

Why it matters: react-hook-form already tracks the value internally (that's the point — fewer re-renders). Don't shadow it with your own `useState`. If you truly need to *watch* a value, use `watch("name")` instead of a second state.

Heads-up: schema validation with **zod** + **@hookform/resolvers** is a nice next step (define rules in one schema instead of inline), and building your own reusable `<Input>` / `<Select>` wrappers is **Day 7**. Global state with Context/reducers is **Day 8**. For today, inline validation rules and these fixes are all you need.

4 Concept Notes — Recall & Interview (copy by pen)

Copy these short bullets by pen. They are for quick recall and interview answers — routing and forms in React + TypeScript.

Part A — Routing (React Router)

Client-side routing (SPA)

Definition:

- Choosing which screen to show happens in the browser, not on the server.
- SPA = Single Page Application: one HTML page, and JavaScript swaps the content.
- C#/.NET: like ASP.NET routing, but the "route table" lives in the browser, not on the server.

✅ Advantages:

- Fast — no full page reload, only the changed part re-renders.
- Feels smooth, like a desktop app.

❌ Disadvantages / watch-outs:

- First load can be heavier (browser downloads the JS bundle once).
- SEO and deep links need extra setup.

💡 Interview tip:

- Classic Q: client vs server routing. Say: server routing = each URL asks the server for a whole new HTML page (full reload); client routing = JS changes the view in-browser, no reload.

📄 Example:

- `/employees → /employees/5` — changes view without reloading the page.

React Router

Definition:

- The most popular library that adds client-side routing to React.
- C#/.NET: like your MVC route config, but written as JSX components.

✅ Advantages:

- Maps URLs to components for you.
- Gives ready-made hooks + components (Routes, Route, Link, useParams...).

❌ Disadvantages / watch-outs:

- Wrap the whole app once in `<BrowserRouter>` (it is the piece that watches the URL).

📄 Example:

- `<BrowserRouter><App /></BrowserRouter>` — turns routing on for the whole app.

Routes and Route

Definition:

- `<Routes>` picks the one best-matching child; each `<Route>` maps a `path` to an `element` (the component to show).
- C#/.NET: `<Route path="employees" ... />` ~ one entry in your MVC route table.

✓ Advantages:

- Declarative — you read the URL map like a simple list.
- Only the matched route renders.

✗ Disadvantages / watch-outs:

- v6/v7 use `element={<Comp />}`, not the old `component={Comp}`.

📄 Example:

- `<Route path="/employees" element={<EmployeeList />} />` — URL /employees shows the list.

Link vs NavLink

Definition:

- `<Link>` navigates without a page reload.
- `<NavLink>` is a Link that also knows when it is "active" (the current page) so you can style it.

✓ Advantages:

- No reload → keeps app state alive.
- NavLink auto-marks the active menu item for styling.

✗ Disadvantages / watch-outs:

- Never use a plain `<a href>` for internal links — it does a full reload and loses state.

💡 Interview tip:

- Classic Q: Link vs `<a>`. Link = SPA navigation (no reload); `<a>` = full browser reload.

📄 Example:

- `<NavLink to="/employees" className={({isActive}) => isActive ? "on" : ""}>` — highlights the active menu item.

Nested routes + Outlet

Definition:

- A route placed inside another route. The parent shows a shared layout; `<Outlet />` marks the spot where the child renders.
- C#/.NET: `<Outlet />` ~ `@RenderBody()` inside a `_Layout.cshtml`.

✓ Advantages:

- Share one layout (nav / sidebar) across many pages.
- Less repeated code.

✗ Disadvantages / watch-outs:

- Forget `<Outlet />` and the child page never appears.

📄 Example:

- `<Route element={<Layout/>}><Route path=":id" element={<Detail/>} /></Route>` — Detail renders inside Layout's Outlet.

Index routes

Definition:

- The default child shown when the parent path matches exactly (no extra segment).

✓ Advantages:

- Clean way to show a "home / default" view for a section.

✗ Disadvantages / watch-outs:

- Use the `index` keyword, not a `path`; it has no path segment of its own.

📄 Example:

- `<Route index element={<Dashboard/>} />` — shows Dashboard at the parent's URL.

Route params (useParams)

Definition:

- A changing part of the URL (like an id); `useParams()` reads it inside the component.
- C#/.NET: `:id` ~ the `{id}` route parameter in ASP.NET.

✓ Advantages:

- One route handles many records (`/employees/1`, `/employees/2...`).

✗ Disadvantages / watch-outs:

- In TS, a param is `string | undefined` — convert to number and check before using it.

💡 Interview tip:

- Classic Q: how do route params work? You declare `:id` in the path; the router matches the URL and hands you `{ id }` via `useParams`.

📄 Example:

- `const { id } = useParams<{ id: string }>()` — reads the `:id` from the URL.

Programmatic navigation (useNavigate)

Definition:

- Move to another route from code (not from a click) — e.g. after saving a form.
- C#/.NET: `navigate("/employees")` ~ `return RedirectToAction("Index")`.

✅ Advantages:

- Redirect after an action (save, login, delete).
- Can go back/forward: `navigate(-1)`.

❌ Disadvantages / watch-outs:

- Call `navigate()` inside an event or effect, never during render.

📄 Example:

- `navigate("/employees")` — jumps to the list after saving.

Query params (useSearchParams)

Definition:

- The `?key=value` part of the URL (search, filter, page number); `useSearchParams()` reads and updates it.
- C#/.NET: `?page=2` ~ `Request.Query["page"]`.

✅ Advantages:

- Shareable / bookmarkable — the URL holds the search or filter.
- Works like `useState`, but the value is stored in the URL.

❌ Disadvantages / watch-outs:

- Values are always strings (convert if you need a number).

📄 Example:

- `const [sp] = useSearchParams(); sp.get("q")` — reads `?q=` from the URL.

Not-found route (404)

Definition:

- A catch-all route using `path="*"` that shows when no other route matches.
- C#/.NET: `path="*" ~ a catch-all / 404 fallback route.`

✅ Advantages:

- Friendly "page not found" instead of a blank screen.

❌ Disadvantages / watch-outs:

- Put it last — `*` matches anything.

📄 Example:

- `<Route path="*" element={<NotFound/>} />` — shows for any unknown URL.

Part B — Forms (React Hook Form)

React Hook Form (RHF)

Definition:

- A small library that manages form values, validation, and submit with very little code.
- C#/.NET: RHF validation ~ data annotations / model validation on a view model.

✅ Advantages:

- Less code, and fast (very few re-renders).
- Built-in validation + error messages.

❌ Disadvantages / watch-outs:

- Custom UI components may need `<Controller>` (see below).

💡 Interview tip:

- Classic Q: why RHF? It is uncontrolled (uses refs), so typing does not re-render the whole form → better performance and less code than manual `useState`.

📄 Example:

- ```
const { register, handleSubmit, formState:{errors} } = useForm<FormValues>()
```

 — sets up the form.

### register

#### Definition:

- Connects an input to RHF — tells it to track this field's value and its rules.

#### ✅ Advantages:

- One line wires up value tracking + validation.

#### ❌ Disadvantages / watch-outs:

- Spread it onto the input: 

```
{ ...register("name") }
```

.

#### 📄 Example:

- ```
<input { ...register("email", { required: true }) } />
```

 — tracks email + makes it required.

handleSubmit

Definition:

- Wraps your submit function: it validates first, then calls your function only if the form is valid.

✅ Advantages:

- Runs validation for you and gives clean, typed data.
- Blocks submit when the form is invalid.

📄 Example:

- `<form onSubmit={handleSubmit(onSave)}>` — `onSave` runs only if the form is valid.

formState.errors

Definition:

- An object holding the validation errors per field, used to show messages.

✓ Advantages:

- Easy per-field error messages.
- Auto-updates as the user fixes each field.

📄 Example:

- `{errors.email} && <span>{errors.email.message}</span>}` — shows the email error.

Form validation (rules + schema / zod)

Definition:

- Rules that decide if a value is OK — either inline rules per field, or one schema (a single object describing all rules).
- C#/.NET: `required`, `minLength` ~ `[Required]`, `[MinLength]` data annotations; a zod schema ~ a validated view model.

✓ Advantages:

- Inline rules = quick. Schema (zod) = one place, reusable, strong types.

✗ Disadvantages / watch-outs:

- For zod you add a resolver from `@hookform/resolvers` (light mention — deeper later).

📄 Example:

- `register("age", { required: "Required", min: { value: 18, message: "18+" } })` — inline rules with messages.
- `resolver: zodResolver(schema)` — validate the whole form from one schema.

Controlled vs uncontrolled forms

Definition:

- Controlled = React state holds each value (`value` + `onChange`).
- Uncontrolled = the DOM holds the value; you read it only when needed (via a ref).
- C#/.NET (Knockout): controlled ~ a two-way `value` binding that updates the observable on each keystroke.

✓ Advantages:

- Controlled: full control, easy live formatting.
- Uncontrolled: less code, fewer re-renders (faster).

✗ Disadvantages / watch-outs:

- Mixing both on one input causes React warnings.

💡 Interview tip:

- Classic Q: controlled vs uncontrolled. Controlled = value lives in state, re-renders each keystroke; uncontrolled = value lives in the DOM/ref, read on demand. RHF is uncontrolled → that is its speed win.

📄 Example:

- `<input value={v} onChange={e⇒setV(e.target.value)} />` — controlled: state updates per keystroke.

Controller

Definition:

- A wrapper that lets RHF work with controlled / custom components (date pickers, selects, MUI) that do not expose a plain ref.

✅ Advantages:

- Bridges RHF to any custom input.
- Keeps validation + errors working.

✗ Disadvantages / watch-outs:

- Use it only when `register` can't attach; plain HTML inputs don't need it.

📄 Example:

- `<Controller name="dept" control={control} render={({field}) ⇒ <MySelect { ... field} /> />` — wires a custom select into RHF.